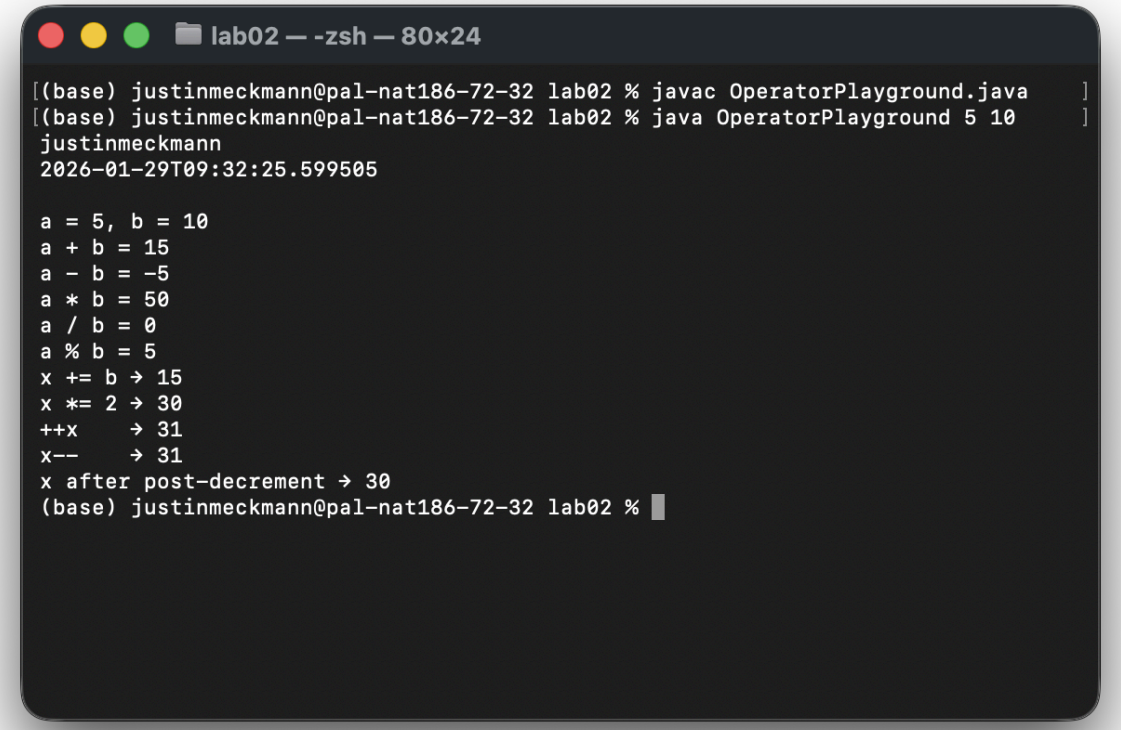
Justin Meckmann | 039582456
jmeckman@purdue.edu

CNIT255 Lab 02 Part A

I. Operators

OperatorPlayground.java: Output



2. Explain why 15 / 2 prints 7 and 15 % 2 prints 1.

- 15 / 2 prints 7 because both values are int, so Java uses integer division and truncates the decimal part.
- 15 % 2 prints 1 because the modulo operator returns the remainder of the division

3. Explain and provide a code snippet demonstrating how you could force decimal division without changing the input values.

Decimal division can be forced by casting the result variable to a floating-point type before dividing.

```
int a = 15;
int b = 2;
double result = (double) a / b;
System.out.println(result); // 7.5
```

II. Casting Demo

CastingDemo.java: Output

```
lab02 — -zsh — 80×24
[(base) justinmeckmann@pal-nat186-72-32 lab02 % javac CastingDemo.java
[(base) justinmeckmann@pal-nat186-72-32 lab02 % java CastingDemo
 justinmeckmann
 2026-01-29T09:50:31.496011

 automatic: 1134.0
 automatic: 12
 explicit (double->int): 9
 explicit (long->short): -31072
 explicit (float->byte): -126
 explicit (int->char): A
 explicit (double->byte): 2
 (base) justinmeckmann@pal-nat186-72-32 lab02 %
```

Automatic (widening) conversions

- int → double is widening: 1134 can be represented exactly as a double, so value stays 1134.0
- byte → int is widening: 12 fits in int, so value stays 12

Explicit (narrowing) conversions - risk of data loss

- int cannot represent non whole numbers, so that part after the decimal point is truncated (dropped)
- max value of short is 32767 which is less than 100000. Therefore there is an overflow error

float to byte:

- byte cannot represent decimal numbers, so everything after the decimal point is dropped which results in 130
- The max value of byte is 127, which is less than 130, which then results in an overflow error

int to char:

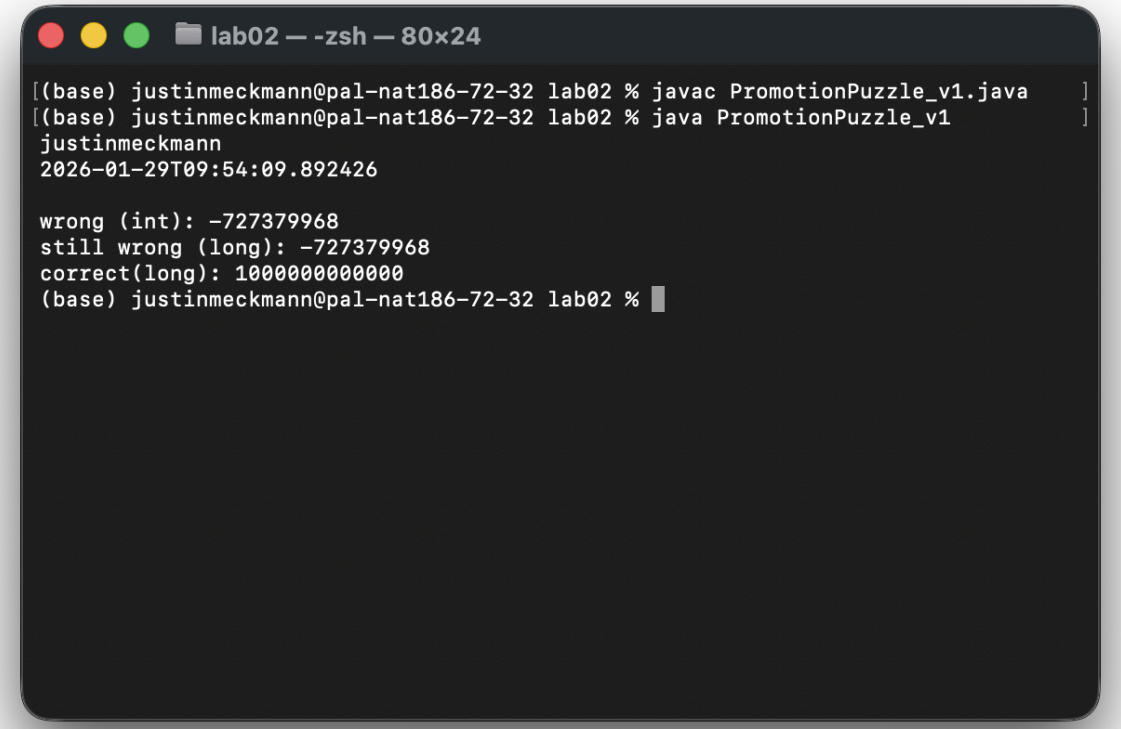
- the corresponding ascii character for the number (e.g. 65 -> A, 66 -> B) is returned

double to byte:

- same as float to byte: truncated and overflow error. All bits but the lowest 8 are dropped

III. Promotion Puzzle

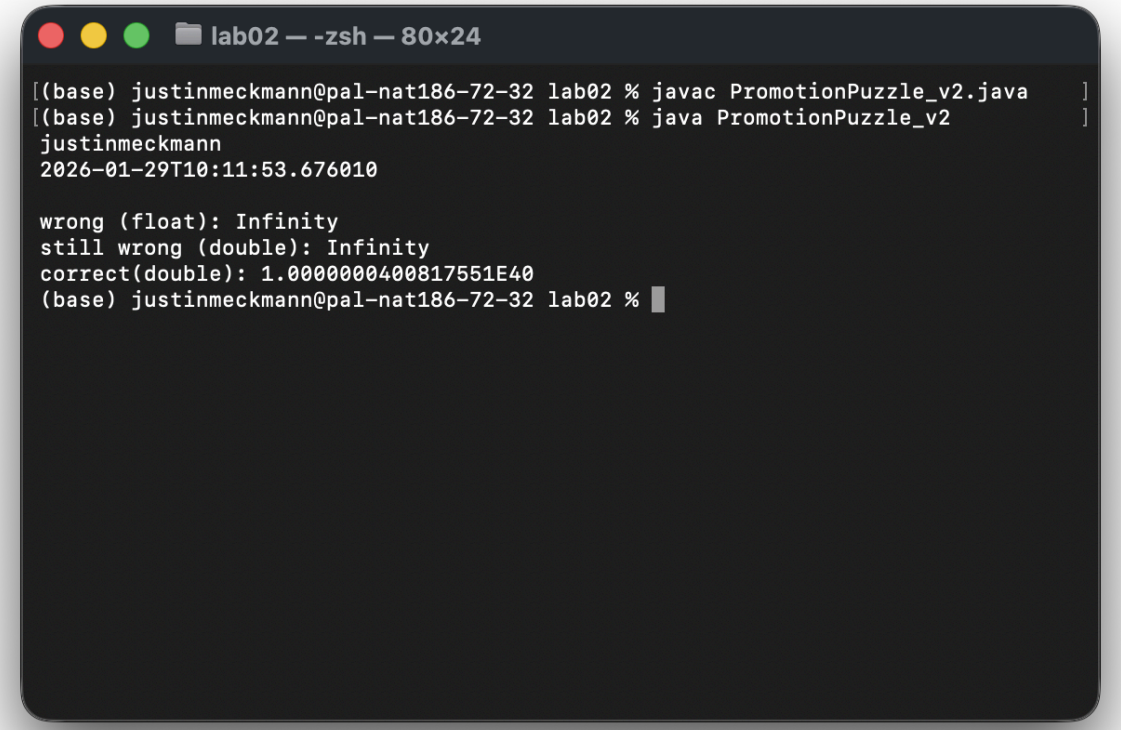
PromotionPuzzle_v1.java Output



3. Why is stillWrong negative?

- Because the operation big*big is done in “integer” math which results in an overflow.
- Because the operation big*big is done in “float” math which results in an overflow

4. Which operand was widened to fix it?

- There was a long operand added, forcing long math

PromotionPuzzle_v2.java Output